

1. 数据挖掘(Data Mining)实验课I实验报告

0.1. 实验工具

VSCode, Jupyter Notebook, python3.10.1

0.2. 分析说明

$$L(\Theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

使用均方误差作为损失函数的优点包括：

- 它对异常值非常敏感，因为平方会放大误差。
- 它是不同误差的连续度量，适合分析模型性能。
- 它的导数（即梯度）在数学上容易处理，便于使用梯度下降等优化算法，作为损失函数。

梯度是损失函数关于参数的偏导数，表示损失函数在参数空间中的变化方向。

对参数的梯度计算如下：

$$\frac{\partial L}{\partial \Theta_{c_2}} = \frac{L(\Theta_A, \Theta_B, \Theta_{c_2} + \Delta, \Theta_{c_1}, \Theta_{c_0}) - L(\Theta_A, \Theta_B, \Theta_{c_2}, \Theta_{c_1}, \Theta_{c_0})}{\Delta}$$

$$\frac{\partial L}{\partial \Theta_{c_1}} = \frac{L(\Theta_A, \Theta_B, \Theta_{c_2}, \Theta_{c_1} + \Delta, \Theta_{c_0}) - L(\Theta_A, \Theta_B, \Theta_{c_2}, \Theta_{c_1}, \Theta_{c_0})}{\Delta}$$

$$\frac{\partial L}{\partial \Theta_{c_0}} = \frac{L(\Theta_A, \Theta_B, \Theta_{c_2}, \Theta_{c_1}, \Theta_{c_0} + \Delta) - L(\Theta_A, \Theta_B, \Theta_{c_2}, \Theta_{c_1}, \Theta_{c_0})}{\Delta}$$

$$\frac{\partial L}{\partial \Theta_A} = \frac{L(\Theta_A + \Delta, \Theta_B, \Theta_{c_2}, \Theta_{c_1}, \Theta_{c_0}) - L(\Theta_A, \Theta_B, \Theta_{c_2}, \Theta_{c_1}, \Theta_{c_0})}{\Delta}$$

$$\frac{\partial L}{\partial \Theta_B} = \frac{L(\Theta_A, \Theta_B + \Delta, \Theta_{c_2}, \Theta_{c_1}, \Theta_{c_0}) - L(\Theta_A, \Theta_B, \Theta_{c_2}, \Theta_{c_1}, \Theta_{c_0})}{\Delta}$$

0.3. 关键代码

关键代码:以下是实验关键代码,用于梯度下降更新参数:

```
1 # 预测函数
2 def predict_f(INPUT_X, THETA_A, THETA_B, THETA_c2, THETA_c1, THETA_c0):
3     OUTPUT_Y = THETA_A * np.sin(THETA_c2 * INPUT_X ** 2 + THETA_c1 * INPUT_X
4     + THETA_c0) + THETA_B
5     return OUTPUT_Y
6
7 # 损失函数
8 def compute_loss(y_true, y_pred):
9     return np.mean((y_true - y_pred) ** 2)
```

```

9
10 # 当前总体损失
11 def loss_all(DATA_X, DATA_Y, Theta):
12     y_pred = predict_f(DATA_X, *Theta)
13     return compute_loss(DATA_Y, y_pred)
14
15 # 计算梯度的数值方法
16 def compute_gradient(DATA_X, DATA_Y, Theta, DELTA=0.1):
17     num_params = len(Theta)
18     gradients = np.zeros(num_params)
19     for i in range(num_params):
20         Theta_plus = Theta.copy()
21         Theta_plus[i] += DELTA
22         L_plus = loss_all(DATA_X, DATA_Y, Theta_plus)
23         Theta_minus = Theta.copy()
24         Theta_minus[i] -= DELTA
25         L_minus = loss_all(DATA_X, DATA_Y, Theta_minus)
26         gradients[i] = (L_plus - L_minus) / (2*DELTA)
27     return gradients
28
29 for epoch in range(EPOCHES):
30     epoch_loss = 0 # 初始化epoch损失
31     gradients = compute_gradient(DATA_X, DATA_Y, Theta) # 计算梯度
32     for i in range(len(DATA_X)):
33         # 计算预测值和损失
34         y_pred = predict_f(DATA_X[i], *Theta)
35         current_loss = compute_loss(DATA_Y[i], y_pred)
36         print("Parameters =", Theta)
37         print("Loss Value =", epoch_loss)
38         print("Gradients =", gradients)
39         # 累加损失
40         epoch_loss += current_loss
41     # 更新参数
42     Theta = [t - ALPHA * g for t, g in zip(Theta, gradients)]
43     # 计算平均损失
44     # epoch_loss /= len(DATA_X)
45     loss_values.append(epoch_loss)
46     theta_history.append(Theta[:]) # 保存当前参数
47     # 检查是否是损失值最小的迭代
48     if epoch_loss < min_loss:
49         min_loss = epoch_loss
50         min_loss_epoch = epoch

```

0.4. 实验结果:

以下是实验结果的输出,关于训练过程中的参数和梯度变化:

```

1 ===== EPOCH 12 Average Loss Value = 272.4630 =====
2 Parameters = [np.float64(7.092399217614069), np.float64(21.39327015538108),
3 np.float64(-8.5898171319425), np.float64(182.97970991119092),
4 np.float64(49.75782273213084)]
5 Loss Value = 0
6 Gradients = [ 8.3468929 -2.22917802 -13.7640559 -130.92367143
7 -22.99125283]

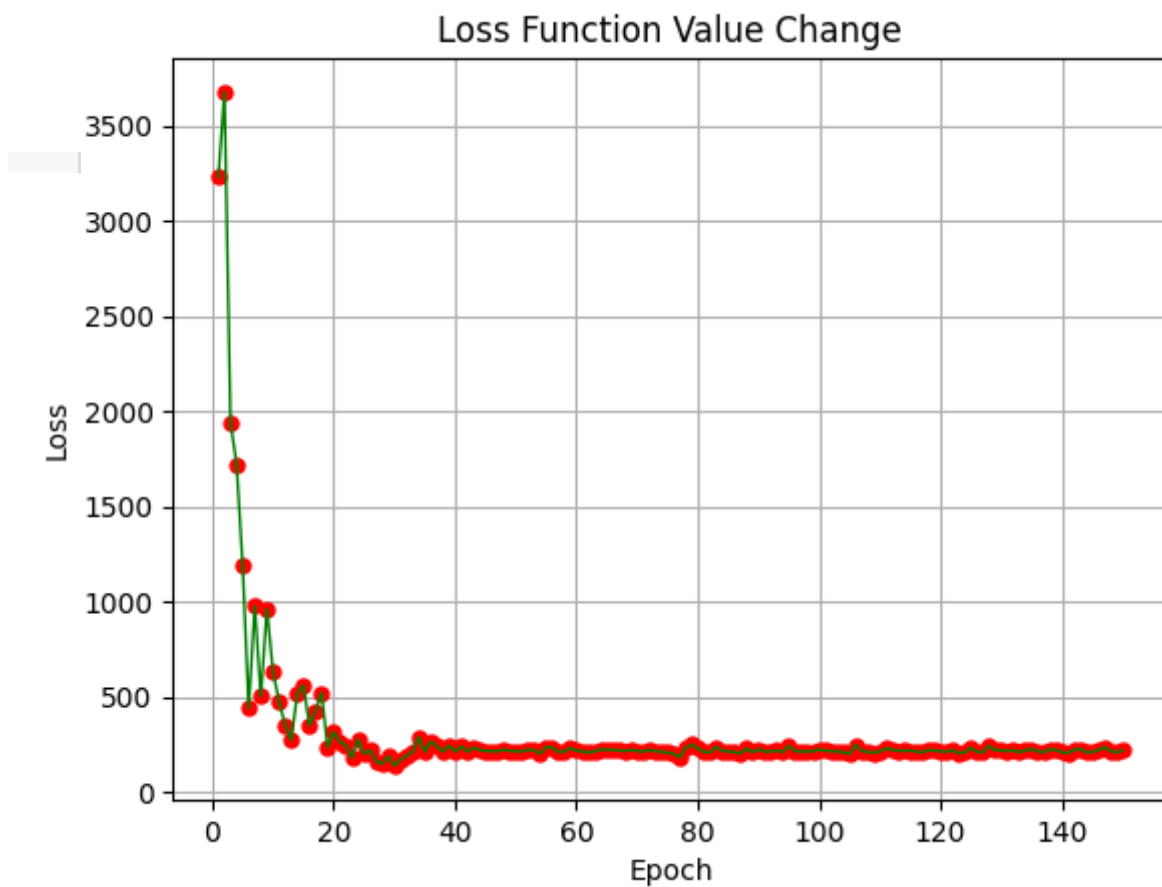
```

```

5 Parameters = [np.float64(7.092399217614069), np.float64(21.39327015538108),
  np.float64(-8.5898171319425), np.float64(182.97970991119092),
  np.float64(49.75782273213084)]
6 Loss Value = 54.9108348084052
7 Gradients = [ 8.3468929 -2.22917802 -13.7640559 -130.92367143
  -22.99125283]
8 Parameters = [np.float64(7.092399217614069), np.float64(21.39327015538108),
  np.float64(-8.5898171319425), np.float64(182.97970991119092),
  np.float64(49.75782273213084)]
9 Loss Value = 144.77995139863503
10 Gradients = [ 8.3468929 -2.22917802 -13.7640559 -130.92367143
  -22.99125283]
11 Parameters = [np.float64(7.092399217614069), np.float64(21.39327015538108),
  np.float64(-8.5898171319425), np.float64(182.97970991119092),
  np.float64(49.75782273213084)]
12 Loss Value = 279.9454567582626
13 Gradients = [ 8.3468929 -2.22917802 -13.7640559 -130.92367143
  -22.99125283]
14 Parameters = [np.float64(7.092399217614069), np.float64(21.39327015538108),
  np.float64(-8.5898171319425), np.float64(182.97970991119092),
  np.float64(49.75782273213084)]
15 Loss Value = 333.3328438024874
16 Gradients = [ 8.3468929 -2.22917802 -13.7640559 -130.92367143
  -22.99125283]
17 Parameters = [np.float64(7.092399217614069), np.float64(21.39327015538108),
  np.float64(-8.5898171319425), np.float64(182.97970991119092),
  np.float64(49.75782273213084)]
18 Loss Value = 364.4403176043304
19 Gradients = [ 8.3468929 -2.22917802 -13.7640559 -130.92367143
  -22.99125283]
20 Parameters = [np.float64(7.092399217614069), np.float64(21.39327015538108),
  np.float64(-8.5898171319425), np.float64(182.97970991119092),
  np.float64(49.75782273213084)]
21 Loss Value = 408.9556338193705
22 Gradients = [ 8.3468929 -2.22917802 -13.7640559 -130.92367143
  -22.99125283]
23 Parameters = [np.float64(7.092399217614069), np.float64(21.39327015538108),
  np.float64(-8.5898171319425), np.float64(182.97970991119092),
  np.float64(49.75782273213084)]
24 Loss Value = 409.63204530862816
25 Gradients = [ 8.3468929 -2.22917802 -13.7640559 -130.92367143
  -22.99125283]
26 Parameters = [np.float64(7.092399217614069), np.float64(21.39327015538108),
  np.float64(-8.5898171319425), np.float64(182.97970991119092),
  np.float64(49.75782273213084)]
27 Loss Value = 458.9719336903233
28 Gradients = [ 8.3468929 -2.22917802 -13.7640559 -130.92367143
  -22.99125283]

```

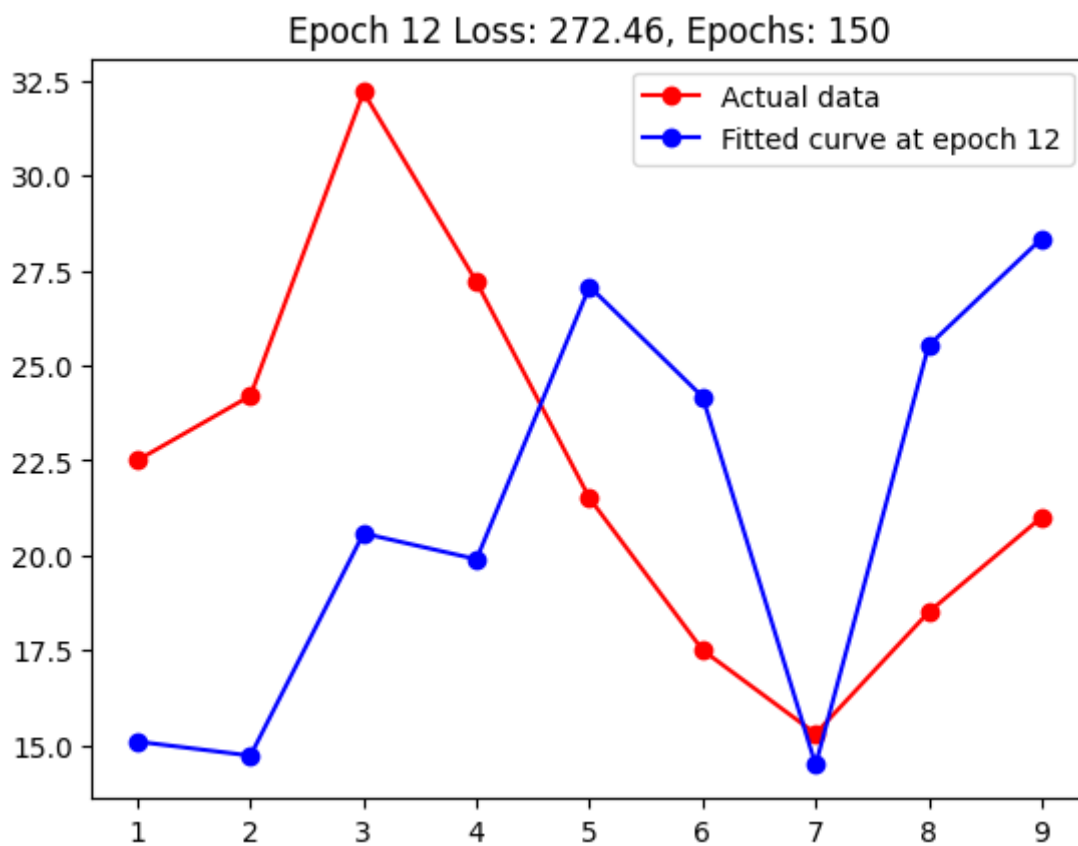
损失函数值变化:



横轴表示训练的轮次，纵轴表示损失值。初始损失值非常高，大约在3500左右。随着训练轮次的增加，损失值迅速下降，并在大约20个轮次后趋于稳定，最终在150个轮次时稳定在272.46左右。

模型在训练初期学习速度很快，损失值显著下降，接近较优的状态。

拟合效果:



实际数据与模型在第12个训练轮次时的拟合曲线，拟合曲线在第3个和第5个数据点附近，拟合曲线与实际数据非常接近，而在第7个和第9个数据点附近，拟合曲线与实际数据的偏差较大。这表明模型在某些区域的预测效果较好，而在其他区域则需要进一步改进。